

MariaDB 데이터 규모와 인덱스 구성에 따른 조회 지연시간 비교

고정 선택도 합성 데이터에서의 반복 측정 연구

작성자 김건호

작성일 2026 년 9 월 30 일

초록

본 연구는 MariaDB 에서 실제 테이블 행 수와 보조 인덱스 구성이 동일한 조회의 지연시간에 미치는 영향을 검증했다. 1 천, 1 만, 10 만, 100 만 행의 합성 데이터를 만들고 보조 인덱스 없음, INDEX(department), INDEX(department, age)의 세 조건을 비교했다. 각 조건에서 3 회 예열 후 10 회 측정하여 총 120 건을 수집했고, 클라이언트가 전체 결과를 받은 시점까지의 중앙값을 사용했다. 100 만 행에서 복합 인덱스는 인덱스 없음보다 중앙값 기준 121.405 ms, 단일 인덱스보다 124.155 ms 짧았다. 반면 단일 인덱스는 100 만 행에서 인덱스 없음보다 2.750 ms 길었으며 실행계획에서도 선택되지 않았다. 따라서 복합 인덱스의 규모 효과와 단일 인덱스 대비 우위는 지지되었지만, 단일 인덱스의 규모 효과는 지지되지 않아 전체 가설은 부분적으로 지지되었다.

주요어 MariaDB 복합 인덱스 조회 지연시간 실행계획 반복 측정

1 서론

1.1 연구 배경과 문제

관계형 데이터베이스에서 인덱스는 검색 범위를 줄일 수 있지만, 인덱스가 존재한다는 사실만으로 항상 짧은 조회시간이 보장되지는 않는다. 데이터 분포, 조회 조건, 반환 행 수와 옵티마이저의 접근 경로 선택이 함께 작용하기 때문이다. 특히 두 열에 동등 조건을 적용하는 조회에서는 첫 번째 열만 포함한 단일 인덱스와 두 열을 함께 포함한 복합 인덱스가 서로 다른 후보 경로를 제공한다. 이 차이를 실제 측정값과 실행계획으로 확인할 필요가 있다.

본 연구는 한 대의 Windows PC 와 MariaDB 11.4.5 환경에서 물리적 테이블 크기를 1 천 행에서 100 만 행까지 바꾸고, 동일한 1 퍼센트 선택도 조회를 반복 측정했다. 이미 알려진 인덱스 동작을 다시 설명하는데 그치지 않고, 이 고정 환경에서 규모별 중앙값 차이와 옵티마이저가 실제로 선택한 접근 경로를 함께 제시하는 것을 목표로 했다.

1.2 연구 질문과 가설

연구 질문은 고정된 합성 데이터 분포와 반복 실행 후 상태에서 실제 테이블 행 수가 증가할 때 보조 인덱스 없음, 단일 인덱스, 복합 인덱스의 조회 지연시간 차이가 어떻게 변하는가이다.

최종 가설은 department 가 DEV 이고 age 가 30 인 조회에서 median_none - median_index 가 100 만 행에서 1 천 행보다 클 것이며, 10 만 행과 100 만 행에서 INDEX(department, age)의 중앙값 지연시간이 INDEX(department)보다 낮을 것이라는 문장으로 고정했다. 앞의 부호 있는 차이가 1 천 행보다 100 만 행에서 커지지 않으면 규모 효과를 지지하지 않고, 복합 인덱스가 10 만 행과 100 만 행 모두에서 단일 인덱스보다 낮지 않으면 우위 가설을 지지하지 않는다는 반증 기준도 측정 전에 정했다. 중간 규모의 비단조 변화와 음수 개선율도 삭제하지 않기로 했다.

1.3 선행 연구

Leis 등(2015)은 실제 데이터와 113 개 다중 조인 질의를 이용해 옵티마이저 구성 요소와 실행 성능을 비교했다. 이 연구는 추정치만으로 실행 성능을 단정하기 어렵고 실제 실행계획을 확인해야 한다는 방법론적 근거를 제공한다. 다만 PostgreSQL 의 다중 조인 연구이므로 본 연구의 MariaDB 단일 테이블 결과를 직접 예측하는 근거로 사용하지 않았다.

MariaDB Documentation(2026)의 Compound Composite Indexes 는 복합 인덱스가 여러 동등 조건을 하나의 B-tree 접근 경로에 포함할 수 있음을 설명해 비교 조건의 구현 근거가 되었다. ANALYZE FORMAT=JSON 문서는 질의를 실제로 실행하고 실제 읽은 행 수와 실행시간을 제공한다고 명시해 측정 10 회 뒤 실행계획을 수집한 근거가 되었다. Query Cache 문서는 SQL_NO_CACHE 가 동일 SELECT 의 결과 캐시 사용을 막는다는 근거를 제공했다. PyMySQL Contributors(2026)의 Cursor Objects 는 execute 의 매개변수 표기와 fetchall 동작을 확인하는 데 사용했다.

선행 연구와 공식 문서는 InnoDB 가 B-tree 보조 인덱스와 복합 인덱스를 제공하고, 옵티마이저가 실행계획을 선택하며, ANALYZE FORMAT=JSON 으로 실제 접근 경로를 관찰할 수 있음을 보여준다. 본 연구는 Windows 와 MariaDB 11.4.5 환경에서 1 퍼센트 선택도의 비커버링 SELECT 를 반복 실행했을 때, 실제 행 수 증가에 따라 none, single, composite 조건의 end-to-end 중앙값 차이가 어떻게 변하는지를 측정한다.

2 연구 방법

2.1 실험 환경

표 1 실험 환경

항목	값
운영체제	Windows-11-10.0.26200-SP0
DBMS	11.4.5-MariaDB
연결 대상	127.0.0.1:33067 / index_study
Python	3.14.6
주요 패키지	PyMySQL 1.2.3, pandas 3.0.6, matplotlib 3.11.2
InnoDB buffer pool	1.98 GiB
Query cache	type OFF, size 1 MiB

2.2 변수와 실험 설계

표 2 변수와 고정 조건

구분	정의	값
독립변수	실제 employees 행 수	1K 10K 100K 1M
독립변수	보조 인덱스 구성	none single composite
종속변수	전체 결과 fetch 까지의 지연시간	ms 중앙값
고정 조건	조회값과 선택도	DEV age 30 정확히 1 퍼센트
반복	조건별 예열과 측정	예열 3 회 측정 10 회
표본 범위	본 실험 측정 건수	4 규모 x 3 조건 x 10 회 총 120 건

합성 데이터는 seed 20260915 로 고정했다. 10 개 부서와 25 세부터 34 세까지의 10 개 나이를 결합한 100 개 조합을 반복해 DEV 와 age 30 의 교집합이 모든 규모에서 정확히 1 퍼센트가 되도록 했다. 규모를 WHERE 절의 id 범위로 흉내 내지 않고 측정 전마다 employees 테이블을 정확한 물리적 행 수로 재구성했다.

2 3 조회와 인덱스 조건

```
SELECT SQL_NO_CACHE id, employee_code, salary, payload
FROM employees
WHERE department = %s AND age = %s
```

매개변수는 DEV 와 30 이다. 반환 열에는 두 보조 인덱스에 포함되지 않은 열을 넣어 복합 인덱스만 covering index 가 되는 혼입을 피했다. none 조건은 PRIMARY KEY 외 보조 인덱스가 없고, single 은 INDEX(department), composite 는 INDEX(department, age)만 존재하도록 각 조건 전에 검사했다.

2 4 실행 및 측정 절차

- 각 규모에서 실제 COUNT 와 DEV 및 age 30 의 분포를 확인했다.
- 사전에 고정된 순서로 인덱스 조건을 구성하고 INFORMATION_SCHEMA 에서 열 순서를 검증했다.
- ANALYZE TABLE 로 옵티마이저 통계를 갱신한 뒤 동일 SELECT 를 3 회 예열했다.
- Python time.perf_counter_ns 로 실행 직전부터 fetchall 완료 직후까지 10 회를 각각 측정했다.
- 측정 종료 뒤 ANALYZE FORMAT=JSON 을 한 번 실행하고 원문 JSON 을 보존했다.
- 원자료, 환경정보, 실행 로그, 요약값, 그래프를 run ID 로 연결하고 SHA-256 manifest 를 생성했다.

2 5 분석과 판정 규칙

각 규모와 인덱스 조건의 대표값은 10 회 지연시간의 중앙값으로 정했다. 부호 있는 차이는 baseline 중앙값에서 candidate 중앙값을 뺀 값이며 양수이면 candidate 가 빨랐음을 뜻한다. 개선율은 이 차이를 baseline 중앙값으로 나눈 비율이다. 1K 보다 1M 에서 none 대비 차이가 커지는지를 single 과 composite 에 각각 확인했다. composite 우위는 100K 와 1M 모두에서 composite 중앙값이 single 보다 낮은지를 기준으로 판정했다. 반복 측정은 독립 표본이 아니므로 유의성 검정이나 모집단 인과 추론은 하지 않았다.

2 6 재현 자료

재현 패키지에는 원자료와 분석 결과, 환경 및 실행 로그, 실험과 분석 스크립트, 실행계획, 그래프, SHA-256 매니페스트가 포함된다. 모든 산출물은 run ID 20260930T080820463265Z 로 연결되며, 실행 순서와 확인 기준은 패키지의 실험 실행 가이드에 기록했다.

3 결과

3 1 원자료 완결성

본 실험 run ID 는 20260930T080820463265Z 이다. 원자료는 120 건이며 12 개 규모-인덱스 조합마다 repeat 1 부터 10 까지가 한 번씩 존재했다. 일치 행 수는 1K 에서 10, 10K 에서 100, 100K 에서 1,000, 1M 에서 10,000 으로 모두 실제 행 수의 1 퍼센트였다. 최종 manifest 는 원자료, 환경정보, 로그, 통계, 그래프와 실행계획 JSON 12 개를 포함한 21 개 파일의 크기와 SHA-256 을 검증했다.

3 2 규모와 인덱스별 지연시간

표 3 규모와 인덱스 조건별 지연시간

행 수	인덱스	증양값 ms	IQR ms	일치 행
1,000	없음	0.512	0.485 - 0.540	10
1,000	단일	0.351	0.324 - 0.425	10
1,000	복합	0.334	0.214 - 0.419	10
10,000	없음	2.043	1.995 - 2.122	100
10,000	단일	1.126	1.099 - 1.194	100
10,000	복합	0.725	0.722 - 0.741	100
100,000	없음	16.875	16.552 - 17.161	1000
100,000	단일	16.861	16.765 - 17.053	1000
100,000	복합	3.645	3.594 - 4.017	1000
1,000,000	없음	154.715	152.898 - 156.170	10000
1,000,000	단일	157.464	154.125 - 159.496	10000
1,000,000	복합	33.309	31.166 - 34.218	10000

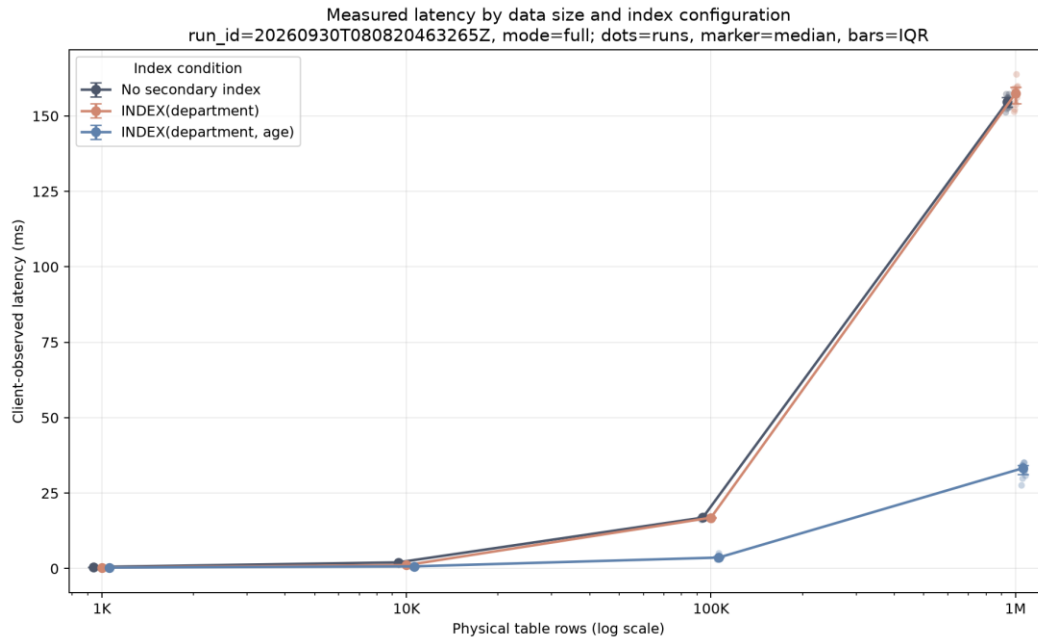


그림 1 데이터 규모와 인덱스 구성별 조회 지연시간

1K 와 10K 에서는 두 인덱스 조건 모두 none 보다 중앙값이 낮았다. 100K 에서는 single 과 none 의 중앙값이 각각 16.861 ms 와 16.875 ms 로 거의 같았지만 composite 는 3.645 ms 였다. 1M 에서는 single 이 157.464 ms 로 none 의 154.715 ms 보다 길었고 composite 는 33.310 ms 로 가장 짧았다.

3.3 부호 있는 차이와 개선율

표 4 중앙값 차이와 개선율

행 수	비교	부호 있는 차이 ms	개선율
1,000	single 대 none	0.161	31.46%
1,000	composite 대 none	0.179	34.89%
1,000	composite 대 single	0.018	5.00%
10,000	single 대 none	0.917	44.88%
10,000	composite 대 none	1.318	64.53%
10,000	composite 대 single	0.401	35.65%
100,000	single 대 none	0.014	0.08%
100,000	composite 대 none	13.230	78.40%
100,000	composite 대 single	13.216	78.38%

행 수	비교	부호 있는 차이 ms	개선율
1,000,000	single 대 none	-2.749	-1.78%
1,000,000	composite 대 none	121.405	78.47%
1,000,000	composite 대 single	124.155	78.85%

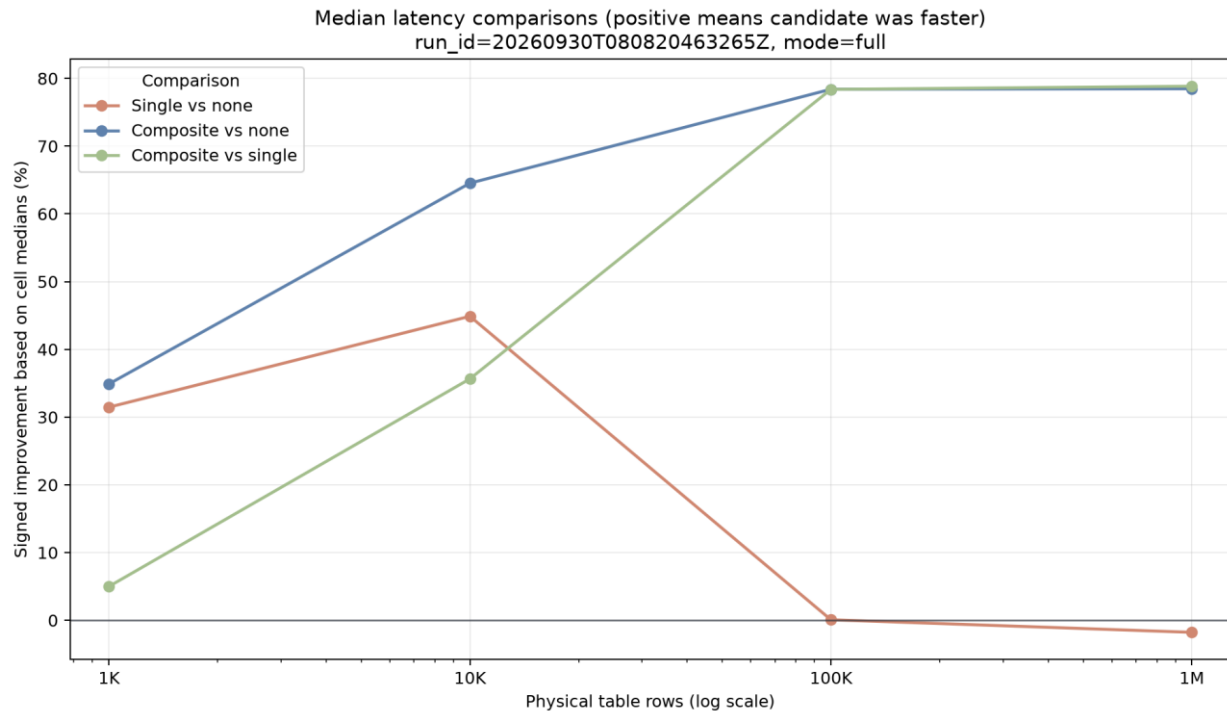


그림 2 규모별 중앙값 개선율

3 4 실행계획

표 5 ANALYZE FORMAT JSON 의 실제 접근 경로

행 수	조건	접근 방식	선택 key	실제 읽은 행
1,000	없음	ALL	사용 안 함	1,000
1,000	단일	ref	idx_employees_department	100
1,000	복합	ref	idx_employees_department_age	10
10,000	없음	ALL	사용 안 함	10,000
10,000	단일	ref	idx_employees_department	1,000
10,000	복합	ref	idx_employees_department_age	100
100,000	없음	ALL	사용 안 함	100,000

행 수	조건	접근 방식	선택 key	실제 읽은 행
100,000	단일	ALL	사용 안 함	100,000
100,000	복합	ref	idx_employees_department_age	1,000
1,000,000	없음	ALL	사용 안 함	1,000,000
1,000,000	단일	ALL	사용 안 함	1,000,000
1,000,000	복합	ref	idx_employees_department_age	10,000

single 조건은 1K 와 10K 에서 idx_employees_department 를 사용했지만 100K 와 1M 에서는 key 를 선택하지 않고 ALL 방식으로 전체 행을 읽었다. department 만으로는 전체의 10 퍼센트를 후보로 찾은 뒤 age 조건을 추가로 검사해야 한다. 규모가 커지자 옵티마이저는 단일 인덱스 접근보다 전체 스캔의 비용을 낮게 판단했다. composite 조건은 모든 규모에서 idx_employees_department_age 를 사용했고 실제 일치 행 수와 같은 수의 행을 반환 경로에서 읽었다.

3 5 가설 판정

표 6 사전 판정 규칙에 따른 가설 평가

가설 부분	근거	판정
single 의 규모 효과	none-single 차이 1K 0.161 ms 1M -2.750 ms	지지되지 않음
composite 의 규모 효과	none-composite 차이 1K 0.179 ms 1M 121.405 ms	지지
composite 의 single 대비 우위	100K 13.216 ms 1M 124.155 ms 짧음	지지

세 판정 중 두 부분은 지지되었고 single 의 규모 효과는 지지되지 않았다. 따라서 최종 가설은 전체 지지가 아니라 부분 지지로 판단한다. 예상과 다른 single 결과는 삭제하거나 0 으로 바꾸지 않았으며 음수 개선율과 실행계획을 함께 보고했다.

4 논의

4 1 복합 인덱스의 규모 효과

복합 인덱스는 두 동등 조건을 모두 key part 로 사용해 실제 일치 행에 가까운 범위로 접근했다. 1M 에서 composite 는 none 보다 약 78.47 퍼센트, single 보다 약 78.85 퍼센트 짧은 중앙값을 보였다.

none 과 composite 의 차이가 1K 의 0.179 ms 에서 1M 의 121.405 ms 로 커졌다는 점은 고정된 1 퍼센트 선택도에서도 물리적 규모가 커질수록 두 접근 경로의 차이가 크게 나타날 수 있음을 보여준다.

4.2 단일 인덱스의 예상 밖 결과

단일 인덱스는 1K 와 10K 에서 선택되었지만 100K 와 1M 에서는 선택되지 않았다. 이때 single 조건은 인덱스가 존재해도 none 과 같은 전체 스캔을 수행했다. 100K 에서 두 중앙값 차이는 0.014 ms 에 불과했고, 1M 에서는 single 이 2.750 ms 더 길었다. 반복 측정 변동과 조건 순서의 영향을 완전히 분리할 수 없으므로 단일 인덱스 자체가 전체 스캔을 느리게 만들었다고 단정할 수는 없다. 다만 이 결과는 인덱스 존재 여부와 실제 사용 여부를 구분해야 한다는 점을 명확히 보여준다.

4.3 연구의 한계

- 한 대의 PC, MariaDB 11.4.5 와 하나의 SELECT 만 측정했으므로 다른 DBMS 와 하드웨어에 일반화할 수 없다.
- 일치 행 수가 데이터 규모의 1 퍼센트로 함께 늘어나므로 탐색 비용과 결과 전송 및 Python materialization 비용이 결합되어 있다.
- 조건별 10 회는 동일 환경의 기술적 반복이며 독립된 서버 표본이 아니다. 통계적 유의성과 모집단 인과를 주장하지 않는다.
- SQL_NO_CACHE 로 query cache 는 배제했지만 InnoDB buffer pool 과 Windows OS cache 는 초기화하지 않아 반복 실행 후 상태를 측정했다.
- 조건 순서는 규모별로 사전 균형화했지만 완전 무작위화하지 않았으므로 잔여 순서 효과가 있을 수 있다.

5 결론

본 연구는 1K 부터 1M 까지의 실제 테이블 행 수와 세 인덱스 구성을 비교해 조회 지연시간 120 건을 측정했다. 복합 인덱스는 모든 규모에서 선택되었고, 1M 에서 중앙값 33.310 ms 로 none 154.715 ms 와 single 157.464 ms 보다 짧았다. 반면 단일 인덱스는 큰 규모에서 선택되지 않았고 규모 효과 가설도 지지되지 않았다. 따라서 최종 가설은 부분 지지로 결론 내리며, 결과의 범위는 고정된 1 퍼센트 선택도와 반복 실행 후 상태의 로컬 MariaDB 환경에 한정된다.

참고문헌

- Leis, V., Gubichev, A., Mirchev, A., Boncz, P., Kemper, A., & Neumann, T. (2015). How Good Are Query Optimizers, Really? Proceedings of the VLDB Endowment, 9(3), 204-215. <https://doi.org/10.14778/2850583.2850594>
- MariaDB Documentation. (2026). ANALYZE FORMAT=JSON. <https://mariadb.com/docs/server/reference/sql-statements/administrative-sql-statements/analyze-and-explain-statements/analyze-format-json>
- MariaDB Documentation. (2026). Compound Composite Indexes. <https://mariadb.com/docs/server/ha-and-performance/optimization-and-tuning/optimization-and-indexes/compound-composite-indexes>
- MariaDB Documentation. (2026). Query Cache. <https://mariadb.com/docs/server/ha-and-performance/optimization-and-tuning/buffers-caches-and-threads/query-cache>
- PyMySQL Contributors. (2026). Cursor Objects. <https://pymysql.readthedocs.io/en/latest/modules/cursors.html>